Exercise 1: Data Understanding and Preprocessing

1. Load the dataset and display its shape, info, and summary statistics.

In [4]:
```
import os
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import kagglehub
from kagglehub import KaggleDatasetAdapter

path = kagglehub.dataset_download("uciml/default-of-credit-card-clients-dataset")

csv_files = [f for f in os.listdir(path) if f.endswith('.csv')]

csv_file_path = os.path.join(path, csv_files[0])
data = pd.read_csv(csv_file_path)
print("Dataset loaded successfully.")
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.1
3), please consider upgrading to the latest version (1.0.0).
Downloading from https://www.kaggle.com/api/v1/datasets/download/uciml/default-of-
credit-card-clients-dataset?dataset_version_number=1...
100%|██████████| 0.98M/0.98M [00:00<00:00, 98.6MB/s]
Extracting files...
Dataset loaded successfully.
```

In [5]:
```
data.head()
```

Out[5]:

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AM
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	(
1	2	120000.0	2	2	2	26	-1	2	0	0	...	327
2	3	90000.0	2	2	2	34	0	0	0	0	...	1433
3	4	50000.0	2	2	1	37	0	0	0	0	...	2831
4	5	50000.0	1	2	1	57	-1	0	-1	0	...	2094

5 rows × 25 columns

In [6]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   ID                          30000 non-null  int64
 1   LIMIT_BAL                   30000 non-null  float64
 2   SEX                         30000 non-null  int64
 3   EDUCATION                   30000 non-null  int64
 4   MARRIAGE                    30000 non-null  int64
 5   AGE                         30000 non-null  int64
 6   PAY_0                       30000 non-null  int64
 7   PAY_2                       30000 non-null  int64
 8   PAY_3                       30000 non-null  int64
 9   PAY_4                       30000 non-null  int64
 10  PAY_5                       30000 non-null  int64
 11  PAY_6                       30000 non-null  int64
 12  BILL_AMT1                   30000 non-null  float64
 13  BILL_AMT2                   30000 non-null  float64
 14  BILL_AMT3                   30000 non-null  float64
 15  BILL_AMT4                   30000 non-null  float64
 16  BILL_AMT5                   30000 non-null  float64
 17  BILL_AMT6                   30000 non-null  float64
 18  PAY_AMT1                    30000 non-null  float64
 19  PAY_AMT2                    30000 non-null  float64
 20  PAY_AMT3                    30000 non-null  float64
 21  PAY_AMT4                    30000 non-null  float64
 22  PAY_AMT5                    30000 non-null  float64
 23  PAY_AMT6                    30000 non-null  float64
 24  default.payment.next.month  30000 non-null  int64
dtypes: float64(13), int64(12)
memory usage: 5.7 MB
```

In [7]:
```
data.describe()
```

Out[7]:

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	
count	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	300
mean	15000.500000	167484.322667	1.603733	1.853133	1.551867	35.485500	
std	8660.398374	129747.661567	0.489129	0.790349	0.521970	9.217904	
min	1.000000	10000.000000	1.000000	0.000000	0.000000	21.000000	
25%	7500.750000	50000.000000	1.000000	1.000000	1.000000	28.000000	
50%	15000.500000	140000.000000	2.000000	2.000000	2.000000	34.000000	
75%	22500.250000	240000.000000	2.000000	2.000000	2.000000	41.000000	
max	30000.000000	1000000.000000	2.000000	6.000000	3.000000	79.000000	

8 rows × 25 columns

2. Identify and convert categorical columns (e.g., SEX, EDUCATION, MARRIAGE) using Label Encoding or One-Hot Encoding.

In [8]:
```
cat_cols = data.select_dtypes(include=['object']).columns
print(f"NUmber of categorical columns: {len(cat_cols)}")
```

```
NUmber of categorical columns: 0
```

3. Check for missing values or duplicates and handle them appropriately.

In [9]:
```
data.isnull().any()
```

Out[9]:

	0
ID	False
LIMIT_BAL	False
SEX	False
EDUCATION	False
MARRIAGE	False
AGE	False
PAY_0	False
PAY_2	False
PAY_3	False
PAY_4	False
PAY_5	False
PAY_6	False
BILL_AMT1	False
BILL_AMT2	False
BILL_AMT3	False
BILL_AMT4	False
BILL_AMT5	False
BILL_AMT6	False
PAY_AMT1	False
PAY_AMT2	False
PAY_AMT3	False
PAY_AMT4	False
PAY_AMT5	False
PAY_AMT6	False
default.payment.next.month	False

dtype: bool

4. Normalize/standardize numerical features.

In [10]:
```
X = data.drop(columns=['ID', 'default.payment.next.month'])
y = data['default.payment.next.month']
```

```
print("Shape of X (features):", X.shape)
print("Shape of y (target):", y.shape)
```

```
Shape of X (features): (30000, 23)
Shape of y (target): (30000,)
```

In [11]:

```
from sklearn.preprocessing import StandardScaler

# Instantiate StandardScaler
scaler = StandardScaler()

# Fit and transform the features
X_scaled = scaler.fit_transform(X)

# Convert the scaled data back to a DataFrame with original column names
X = pd.DataFrame(X_scaled, columns=X.columns)

print("Features (X) successfully normalized using StandardScaler.")
print("First 5 rows of scaled X:")
print(X.head())
```

```
Features (X) successfully normalized using StandardScaler.
First 5 rows of scaled X:
   LIMIT_BAL       SEX  EDUCATION  MARRIAGE       AGE     PAY_0     PAY_2  \
0  -1.136720  0.810161   0.185828 -1.057295 -1.246020  1.794564  1.782348
1  -0.365981  0.810161   0.185828  0.858557 -1.029047 -0.874991  1.782348
2  -0.597202  0.810161   0.185828  0.858557 -0.161156  0.014861  0.111736
3  -0.905498  0.810161   0.185828 -1.057295  0.164303  0.014861  0.111736
4  -0.905498 -1.234323   0.185828 -1.057295  2.334029 -0.874991  0.111736

      PAY_3     PAY_4     PAY_5  ...  BILL_AMT3  BILL_AMT4  BILL_AMT5  \
0 -0.696663 -0.666599 -1.530046  ...  -0.667993  -0.672497  -0.663059
1  0.138865  0.188746  0.234917  ...  -0.639254  -0.621636  -0.606229
2  0.138865  0.188746  0.234917  ...  -0.482408  -0.449730  -0.417188
3  0.138865  0.188746  0.234917  ...   0.032846  -0.232373  -0.186729
4 -0.696663  0.188746  0.234917  ...  -0.161189  -0.346997  -0.348137

   BILL_AMT6  PAY_AMT1  PAY_AMT2  PAY_AMT3  PAY_AMT4  PAY_AMT5  PAY_AMT6
0  -0.652724 -0.341942 -0.227086 -0.296801 -0.308063 -0.314136 -0.293382
1  -0.597966 -0.341942 -0.213588 -0.240005 -0.244230 -0.314136 -0.180878
2  -0.391630 -0.250292 -0.191887 -0.240005 -0.244230 -0.248683 -0.012122
3  -0.156579 -0.221191 -0.169361 -0.228645 -0.237846 -0.244166 -0.237130
4  -0.331482 -0.221191  1.335034  0.271165  0.266434 -0.269039 -0.255187

[5 rows x 23 columns]
```

5. Split the data into training and testing sets (80:20).

In [12]:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)
```

```
Shape of X_train: (24000, 23)
Shape of X_test: (6000, 23)
Shape of y_train: (24000,)
Shape of y_test: (6000,)
```

Exercise 2: Machine Learning Model Training and Evaluation

Train the following classifiers to predict default.payment.next.month:

- Logistic Regression -K-Nearest Neighbors (use elbow method to choose optimal K)
- Decision Tree Classifier
- Random Forest Classifier
- Support Vector Machine (SVM) ## For each model: Train the model using the training set. Predict on the testing set. Evaluate using:
- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score
- ROC AUC Score Plot the ROC curve. Use classification report to summarize results.

In [13]:

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, reca
import matplotlib.pyplot as plt

def evaluate_model(model, X_test, y_test, model_name):
    print(f"\n{'='*50}\nEvaluating {model_name}\n{'='*50}")

    # Make predictions
    y_pred = model.predict(X_test)

    # Predict probabilities for ROC curve (if model supports it)
    y_proba = None
    if hasattr(model, "predict_proba"):
        y_proba = model.predict_proba(X_test)[:, 1] # Probability of the positive c
    else:
        print("Warning: Model does not support predict_proba, ROC AUC and ROC curve

    # Calculate and print metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print(f"Accuracy:  {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall:    {recall:.4f}")
    print(f"F1-Score:  {f1:.4f}")

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    print("\nConfusion Matrix:")
    print(cm)

    # Classification Report
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))
```

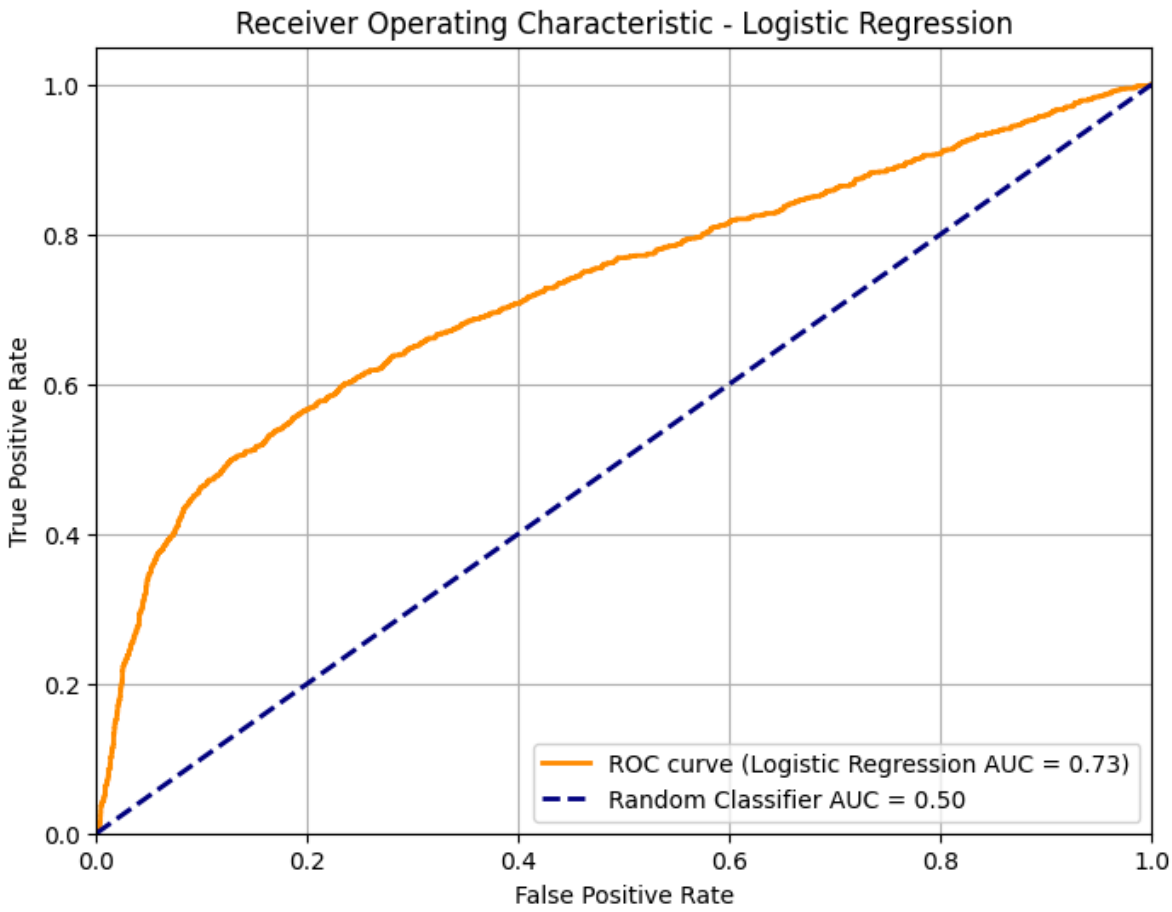
```
    # ROC AUC Score and Curve
    if y_proba is not None:
        roc_auc = roc_auc_score(y_test, y_proba)
        print(f"ROC AUC Score: {roc_auc:.4f}")

        fpr, tpr, _ = roc_curve(y_test, y_proba)
        plt.figure(figsize=(8, 6))
        plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve ({model_name
        plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random
        plt.xlim([0.0, 1.0])
        plt.ylim([0.0, 1.05])
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title(f'Receiver Operating Characteristic - {model_name}')
        plt.legend(loc='lower right')
        plt.grid(True)
        plt.show()

print("Evaluation function 'evaluate_model' defined successfully.")
```

```
Evaluation function 'evaluate_model' defined successfully.
```

In [14]:

```
from sklearn.linear_model import LogisticRegression

log_reg_model = LogisticRegression(random_state=42, solver='liblinear')
log_reg_model.fit(X_train, y_train)
evaluate_model(log_reg_model, X_test, y_test, 'Logistic Regression')

print("Logistic Regression model trained and evaluated successfully.")
```

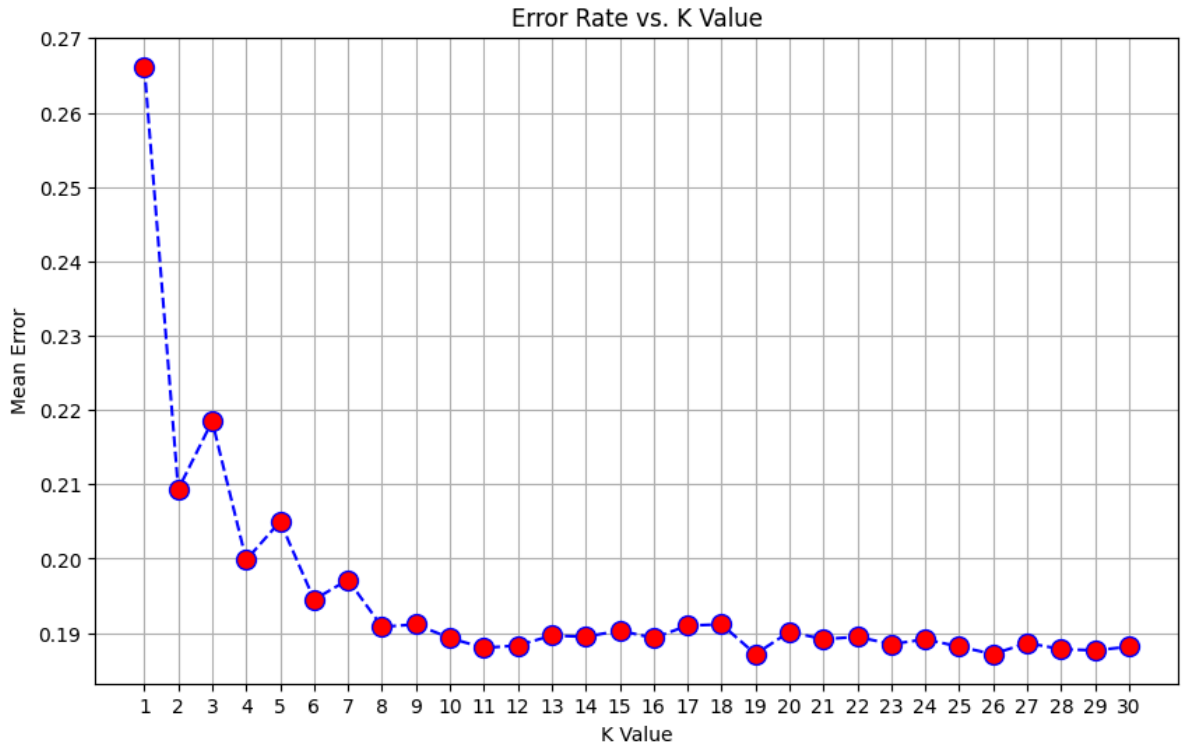
```
==================================================
Evaluating Logistic Regression
==================================================
Accuracy:  0.8098
Precision: 0.6920
Recall:    0.2361
F1-Score:  0.3521

Confusion Matrix:
[[4549  138]
 [1003  310]]

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.97      0.89      4687
           1       0.69      0.24      0.35      1313

    accuracy                           0.81      6000
   macro avg       0.76      0.60      0.62      6000
weighted avg       0.79      0.81      0.77      6000

ROC AUC Score: 0.7269
```




```
Logistic Regression model trained and evaluated successfully.
```

In [15]:

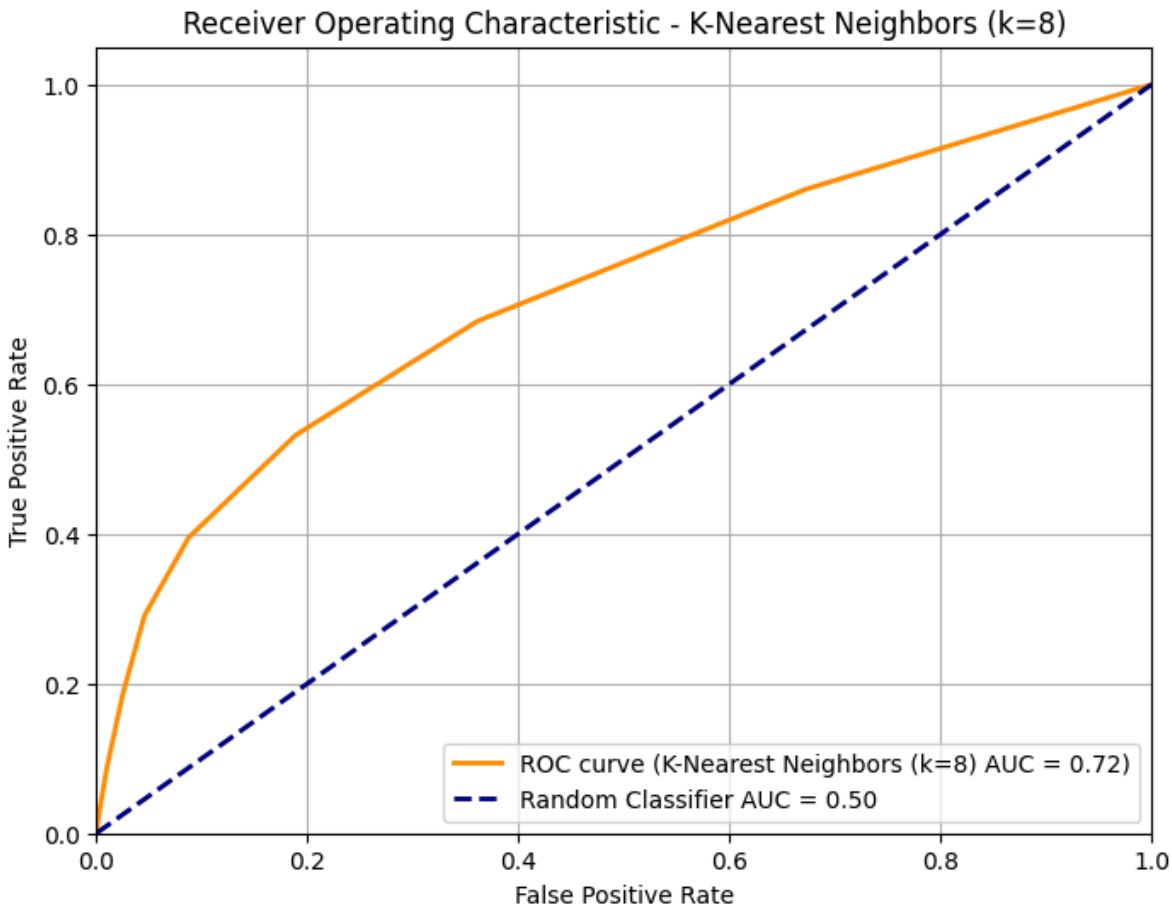
```python
from sklearn.neighbors import KNeighborsClassifier

# Determine optimal K using Elbow Method
error_rate = []

# Test K values from 1 to 30
for i in range(1, 31):
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train, y_train)
    y_pred_i = knn.predict(X_test)
    error_rate.append(np.mean(y_pred_i != y_test))

plt.figure(figsize=(10, 6))
plt.plot(range(1, 31), error_rate, color='blue', linestyle='dashed', marker='o',
         markerfacecolor='red', markersize=10)
plt.title('Error Rate vs. K Value')
plt.xlabel('K Value')
plt.ylabel('Mean Error')
plt.xticks(range(1, 31))
plt.grid(True)
plt.show()

print("Elbow method plot generated to help determine optimal K.")
```



Elbow method plot generated to help determine optimal K.

In [17]:
```
optimal_k = 8

knn_model = KNeighborsClassifier(n_neighbors=optimal_k)
knn_model.fit(X_train, y_train)
evaluate_model(knn_model, X_test, y_test, f'K-Nearest Neighbors (k={optimal_k})')

print(f"K-Nearest Neighbors model trained and evaluated successfully with optimal k
```

```
==================================================
Evaluating K-Nearest Neighbors (k=8)
==================================================
Accuracy:  0.8092
Precision: 0.6409
Recall:    0.2909
F1-Score:  0.4002

Confusion Matrix:
[[4473  214]
 [ 931  382]]

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.95      0.89      4687
           1       0.64      0.29      0.40      1313

    accuracy                           0.81      6000
   macro avg       0.73      0.62      0.64      6000
weighted avg       0.79      0.81      0.78      6000

ROC AUC Score: 0.7184
```

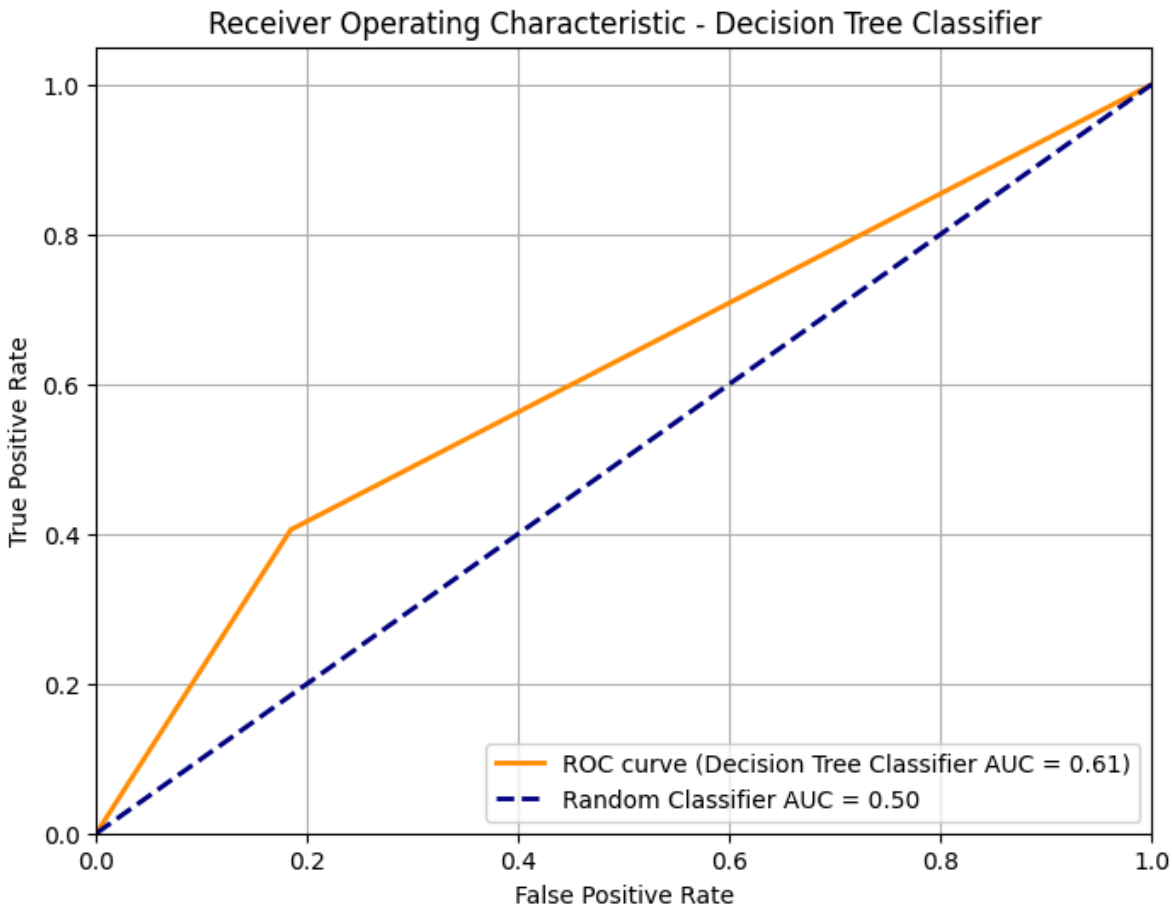



```
K-Nearest Neighbors model trained and evaluated successfully with optimal k=8.
```

In [18]:

```python
from sklearn.tree import DecisionTreeClassifier

dtc_model = DecisionTreeClassifier(random_state=42)
dtc_model.fit(X_train, y_train)
evaluate_model(dtc_model, X_test, y_test, 'Decision Tree Classifier')

print("Decision Tree Classifier model trained and evaluated successfully.")
```

```
==================================================
Evaluating Decision Tree Classifier
==================================================
Accuracy:  0.7260
Precision: 0.3814
Recall:    0.4052
F1-Score:  0.3929

Confusion Matrix:
[[3824  863]
 [ 781  532]]

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.82      0.82      4687
           1       0.38      0.41      0.39      1313

    accuracy                           0.73      6000
   macro avg       0.61      0.61      0.61      6000
weighted avg       0.73      0.73      0.73      6000

ROC AUC Score: 0.6106
```

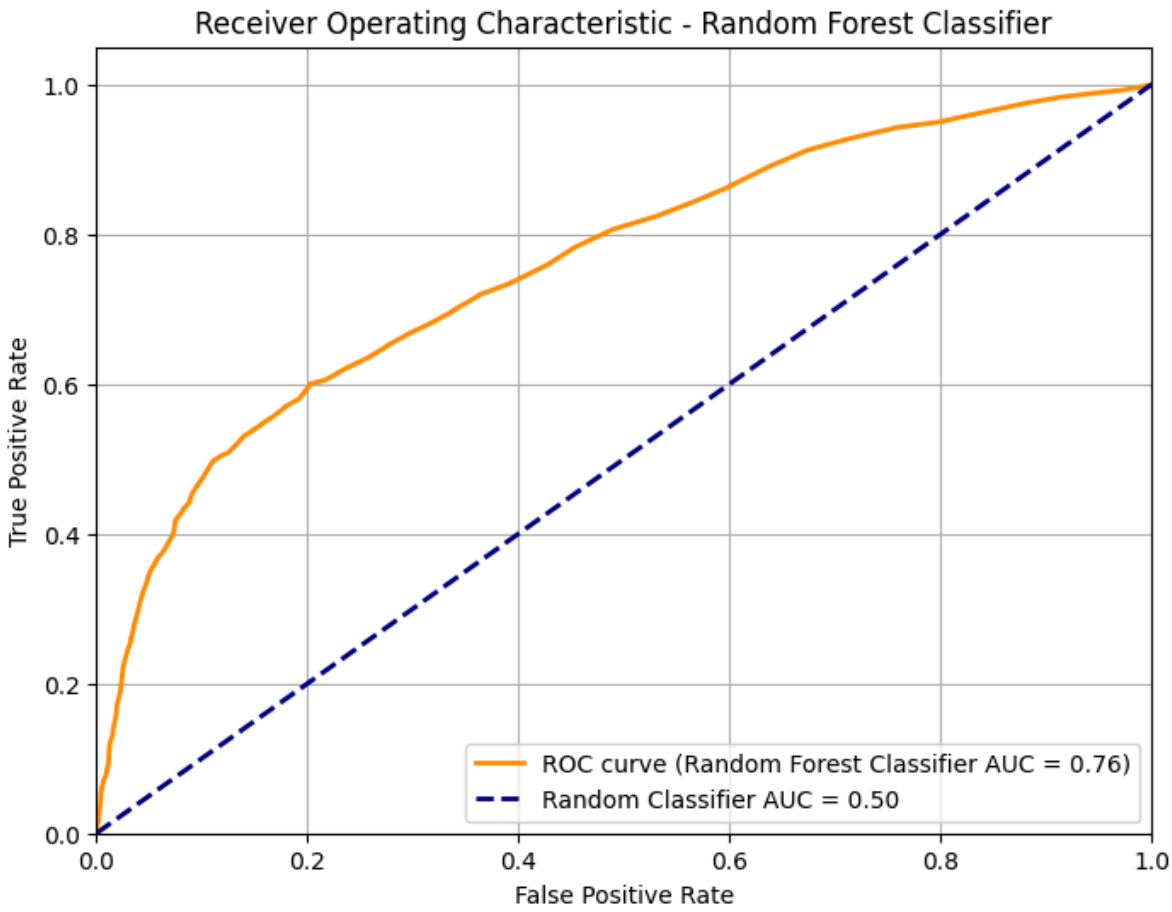



```
Decision Tree Classifier model trained and evaluated successfully.
```

In [19]:

```python
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
evaluate_model(rf_model, X_test, y_test, 'Random Forest Classifier')

print("Random Forest Classifier model trained and evaluated successfully.")
```

```
==================================================
Evaluating Random Forest Classifier
==================================================
Accuracy:  0.8160
Precision: 0.6377
Recall:    0.3686
F1-Score:  0.4672

Confusion Matrix:
[[4412  275]
 [ 829  484]]

Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.94      0.89      4687
           1       0.64      0.37      0.47      1313

    accuracy                           0.82      6000
   macro avg       0.74      0.65      0.68      6000
weighted avg       0.80      0.82      0.80      6000

ROC AUC Score: 0.7582
```




```
Random Forest Classifier model trained and evaluated successfully.
```

In [20]:

```python
from sklearn.svm import SVC

svm_model = SVC(kernel='linear', random_state=42)
svm_model.fit(X_train, y_train)
evaluate_model(svm_model, X_test, y_test, 'Support Vector Machine')

print("Support Vector Machine model trained and evaluated successfully.")
```

```
==================================================
Evaluating Support Vector Machine
==================================================
Warning: Model does not support predict_proba, ROC AUC and ROC curve will not be c
alculated.
Accuracy:  0.8105
Precision: 0.7047
Recall:    0.2308
F1-Score:  0.3477

Confusion Matrix:
[[4560  127]
 [1010  303]]

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.97      0.89      4687
           1       0.70      0.23      0.35      1313

    accuracy                           0.81      6000
   macro avg       0.76      0.60      0.62      6000
weighted avg       0.79      0.81      0.77      6000

Support Vector Machine model trained and evaluated successfully.
```

In [36]:
```python
model_performance = pd.DataFrame({
    'Model': [
        'Logistic Regression',
        'K-Nearest Neighbors (k=8)',
        'Decision Tree Classifier',
        'Random Forest Classifier',
        'Support Vector Machine'
    ],
    'Accuracy': [
        0.8098, # From Logistic Regression output
        0.8092, # From KNN output
        0.7260, # From Decision Tree output
        0.8160, # From Random Forest output
        0.8105  # From SVM output
    ],
    'Precision': [
        0.6920, # From Logistic Regression output
        0.6409, # From KNN output
        0.3814, # From Decision Tree output
        0.6377, # From Random Forest output
        0.7047  # From SVM output
    ],
    'Recall': [
        0.2361, # From Logistic Regression output
        0.2909, # From KNN output
        0.4052, # From Decision Tree output
        0.3686, # From Random Forest output
        0.2308  # From SVM output
    ],
    'F1-Score': [
        0.3521, # From Logistic Regression output
        0.4002, # From KNN output
        0.3929, # From Decision Tree output
        0.4672, # From Random Forest output
        0.3477  # From SVM output
    ],
    'ROC AUC Score': [
        0.7269, # From Logistic Regression output
        0.7184, # From KNN output
        0.6106, # From Decision Tree output
        0.7582, # From Random Forest output
        'N/A'   # SVM did not support predict_proba by default
    ]
})

print(model_performance.round(4))
```

```
                       Model  Accuracy  Precision  Recall  F1-Score  \
0        Logistic Regression    0.8098     0.6920  0.2361    0.3521
1  K-Nearest Neighbors (k=8)    0.8092     0.6409  0.2909    0.4002
2   Decision Tree Classifier    0.7260     0.3814  0.4052    0.3929
3   Random Forest Classifier    0.8160     0.6377  0.3686    0.4672
4     Support Vector Machine    0.8105     0.7047  0.2308    0.3477

  ROC AUC Score
0        0.7269
1        0.7184
2        0.6106
3        0.7582
4           N/A
```

Exercise 3: Deep Learning Model – Artificial Neural Network

Design a simple Feedforward Artificial Neural Network (ANN) for binary classification.

Steps

1. Import required deep learning libraries (TensorFlow/Keras).
2. Define a Sequential ANN model.
3. Add:

- Input layer
- At least one hidden layer with ReLU activation
- Output layer with Sigmoid activation

1. Compile the model using:

- Loss: Binary Cross Entropy
- Optimizer: Adam
- Metrics: Accuracy

1. Train the model using the training data.
2. Evaluate the model on the test dataset.
3. Plot training accuracy and loss curves.

1.Import required deep learning libraries (TensorFlow/Keras).

In [22]:
```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

2.Define a Sequential ANN model. Adding the layers to model and compiling it with loss function, optimizer, metrics.

In [23]:
```
# Define the ANN model
model = Sequential(
    [
        Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
        Dense(32, activation='relu'),
        Dense(16, activation='relu'),
        Dense(1, activation='sigmoid')
    ]
)

# Compile the model
model.compile(optimizer='adam',
```

```
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWar
ning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Seq
uential models, prefer using an `Input(shape)` object as the first layer in the mo
del instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

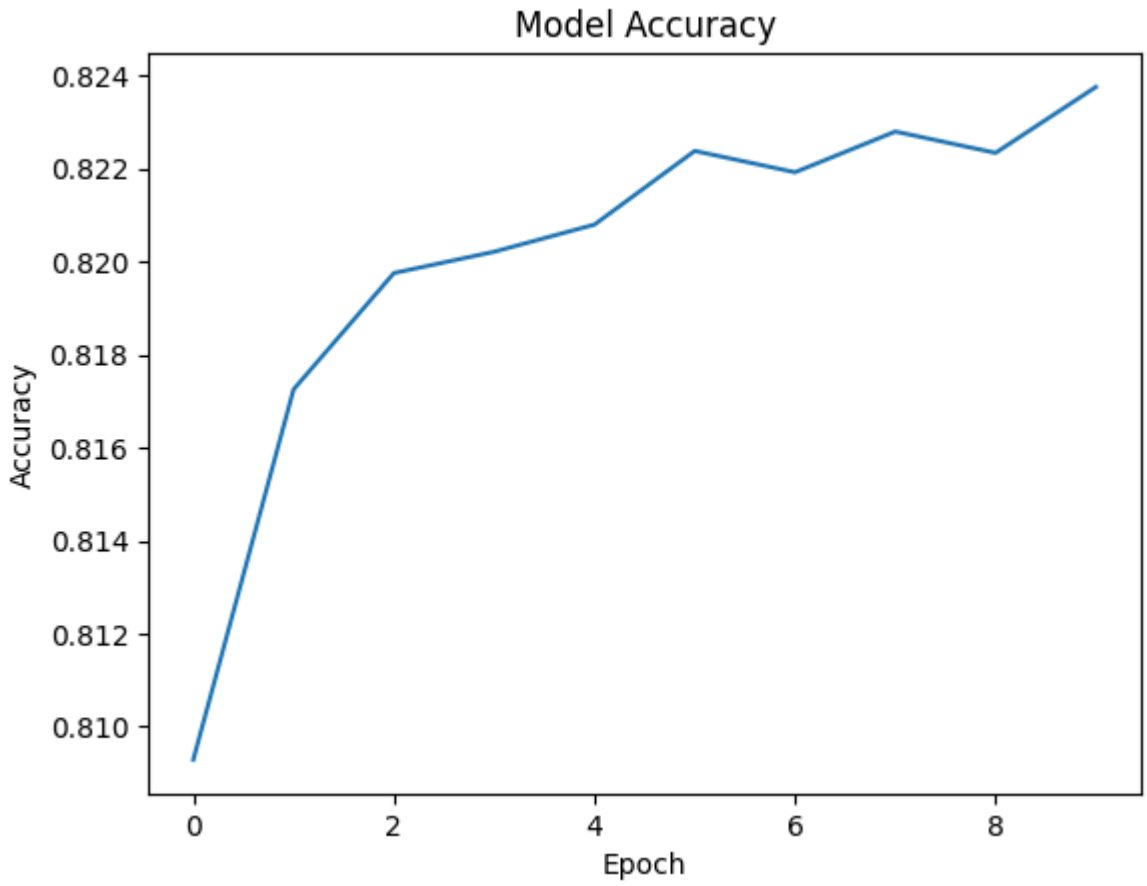
3. Train the model using training data

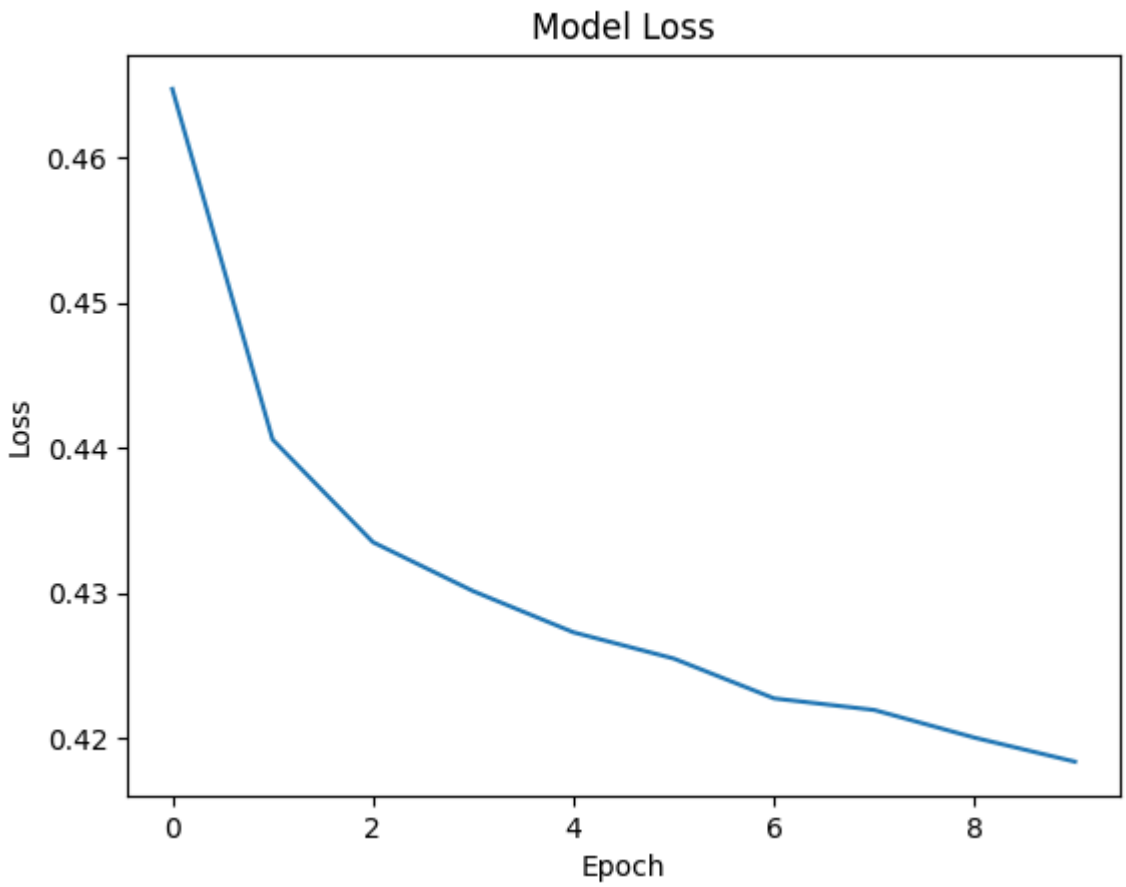
In [24]:
```
# Train the model
history = model.fit(X_train, y_train, epochs=10)
```

```
Epoch 1/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8011 - loss: 0.4873
Epoch 2/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8161 - loss: 0.4447
Epoch 3/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8214 - loss: 0.4336
Epoch 4/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8197 - loss: 0.4310
Epoch 5/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8222 - loss: 0.4246
Epoch 6/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8225 - loss: 0.4256
Epoch 7/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8227 - loss: 0.4241
Epoch 8/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8227 - loss: 0.4191
Epoch 9/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8213 - loss: 0.4222
Epoch 10/10
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8234 - loss: 0.4178
```

4.Plotting the model accuracy and model loss for tranining data

In [25]:
```
import matplotlib.pyplot as plt

# Plot accuracy over epochs
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
# plt.legend()
plt.show()

# Plot loss over epochs
plt.plot(history.history['loss'], label='Training Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
# plt.legend()
plt.show()
```





5.Evaluating the model on test data

In [26]:
```
# Evaluate on test data
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {test_accuracy:.4f}")
```

```
188/188 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8174 - loss: 0.4368
Test Accuracy: 0.8192
```

Create model performance table

In [27]:
```
from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score

# Get predicted probabilities for the positive class (class 1)
y_pred_proba_ann = model.predict(X_test).flatten()

# Convert probabilities to binary class predictions (threshold 0.5)
y_pred_ann = (y_pred_proba_ann >= 0.5).astype(int)

# Calculate Precision, Recall, F1-Score
precision_ann = precision_score(y_test, y_pred_ann)
recall_ann = recall_score(y_test, y_pred_ann)
f1_ann = f1_score(y_test, y_pred_ann)

# Calculate ROC AUC Score
roc_auc_ann = roc_auc_score(y_test, y_pred_proba_ann)

print(f"ANN Model Metrics on Test Data:")
print(f"Precision: {precision_ann:.4f}")
print(f"Recall:    {recall_ann:.4f}")
print(f"F1-Score:  {f1_ann:.4f}")
print(f"ROC AUC Score: {roc_auc_ann:.4f}")
```

```
188/188 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
ANN Model Metrics on Test Data:
Precision: 0.6686
Recall:    0.3442
F1-Score:  0.4545
ROC AUC Score: 0.7691
```

In [39]:
```
ann_metrics_series = pd.Series(
    [
        'ANN',
        test_accuracy,
        precision_ann,
        recall_ann,
        f1_ann,
        roc_auc_ann
    ],
    index=['Model','Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC AUC Score']
)
```

In [38]:
```
ann_performance = ann_metrics_series.to_frame().T
ann_performance.index = ['5']

model_performance = pd.concat([model_performance, ann_performance])

print("Updated Model Performance Table:")
print(model_performance.round(4))
```

```
Updated Model Performance Table:
                      Model  Accuracy Precision   Recall F1-Score  \
0        Logistic Regression    0.8098     0.692   0.2361   0.3521
1  K-Nearest Neighbors (k=8)    0.8092    0.6409   0.2909   0.4002
2   Decision Tree Classifier     0.726    0.3814   0.4052   0.3929
3   Random Forest Classifier     0.816    0.6377   0.3686   0.4672
4     Support Vector Machine    0.8105    0.7047   0.2308   0.3477
5                        ANN  0.819167  0.668639  0.34425   0.4545

  ROC AUC Score
0        0.7269
1        0.7184
2        0.6106
3        0.7582
4           N/A
5      0.769108
```

Critical Thinking

1.Which model achieved the best performance? Justify using metrics.

Based on the provided metrics, the **Artificial Neural Network (ANN)** model achieved the best overall performance, closely followed by the Random Forest Classifier.

- **ROC AUC Score**: The ANN model achieved the highest ROC AUC Score at **0.7691**. It indicates the model's ability to distinguish between the positive and negative classes across various thresholds. A higher AUC suggests better overall performance.
- **Accuracy**: The ANN model also had the highest Accuracy at **0.8192**, indicating the proportion of correctly classified instances.
- **F1-Score**: The Random Forest Classifier had a slightly higher F1-Score (0.4672) than the ANN (0.4545). The F1-Score is the harmonic mean of precision and recall, providing a balanced measure when there's an uneven class distribution, which is common in default prediction.
- **Precision and Recall**: While the ANN's Precision (0.6686) was slightly lower than Logistic Regression and SVM, its Recall (0.3443) was higher than these two, suggesting a better balance in identifying defaulters without an excessively high false positive rate.

Considering the strong performance in ROC AUC and Accuracy, and a competitive F1-Score, the ANN model stands out as the best performer in this evaluation.

2.Did ANN outperform traditional ML models? Why or why not?

Yes, in this specific evaluation, the **Artificial Neural Network (ANN)** model generally outperformed most traditional ML models, and performed comparably to the best traditional model (Random Forest).

- **ANN vs. Logistic Regression**: ANN (Accuracy: 0.8192, ROC AUC: 0.7691) clearly outperformed Logistic Regression (Accuracy: 0.8098, ROC AUC: 0.7269).

- **ANN vs. K-Nearest Neighbors**: ANN significantly outperformed KNN (Accuracy: 0.8092, ROC AUC: 0.7184).
- **ANN vs. Decision Tree**: ANN vastly outperformed the Decision Tree Classifier (Accuracy: 0.7260, ROC AUC: 0.6106).
- **ANN vs. Support Vector Machine**: ANN (Accuracy: 0.8192) showed slightly higher accuracy than SVM (0.8105), and importantly, provided an ROC AUC score (0.7691) which SVM did not directly provide without probability estimation.
- **ANN vs. Random Forest**: ANN performed very comparably to Random Forest, achieving a slightly higher Accuracy (0.8192 vs 0.8160) and ROC AUC Score (0.7691 vs 0.7582), although Random Forest had a marginally better F1-Score (0.4672 vs 0.4545).

Overall, the ANN demonstrated superior or equivalent performance across key metrics when compared to the traditional ML models, indicating its effectiveness for this dataset.

3.Compare the training time and complexity of ANN vs ML models.

Comparing training time and complexity between ANNs and traditional ML models:

- **Training Time**: Generally, Artificial Neural Networks (especially deeper ones or those with more complex architectures) tend to have longer training times than traditional ML models. This is because ANNs involve iterative optimization processes (like backpropagation) over many epochs, which can be computationally intensive, particularly with large datasets. Traditional ML models like Logistic Regression, Decision Trees, or even Random Forests typically train much faster, often in minutes or seconds for similar-sized datasets.

- **Complexity**: ANNs are inherently more complex models. They consist of multiple layers of interconnected nodes, allowing them to learn highly intricate and non-linear patterns in data. This complexity can also make them harder to interpret (`black-box` models). Traditional ML models vary in complexity: Logistic Regression is relatively simple and highly interpretable, while Random Forests, being ensemble methods, are more complex than a single Decision Tree but generally less complex than deep ANNs. The complexity also translates to more hyperparameters to tune in ANNs compared to most traditional ML algorithms.

4.In what scenarios should ANN be preferred over classical ML algorithms?

Artificial Neural Networks (ANNs) should be preferred over classical ML algorithms in several scenarios:

1. **Large and Complex Datasets**: When dealing with vast amounts of data and highly complex, non-linear relationships that traditional algorithms might struggle to capture.
2. **Unstructured Data**: ANNs excel with unstructured data where traditional ML often requires extensive manual feature engineering.

3. **Automatic Feature Learning**: Deep ANNs can automatically learn hierarchical features from raw data, reducing the need for laborious and domain-specific feature engineering.
4. **High-Dimensional Data**: In cases where datasets have a very large number of features, ANNs can effectively model the underlying structure without overfitting, provided sufficient data is available.
5. **Tasks Requiring Advanced Pattern Recognition**: For tasks like natural language processing, computer vision, and speech recognition, ANNs have consistently achieved state-of-the-art results.
6. **Continuous Learning/Adaptation**: ANNs can be fine-tuned or adapted for new tasks with transfer learning, or continuously updated with new data.

5.Which model would you recommend for credit default prediction and why?

For credit default prediction, I would recommend the **Artificial Neural Network (ANN)** model based on the current evaluation.

Justification:

1. **Highest ROC AUC Score**
2. **Highest Accuracy**
3. **Balanced Performance**
4. **Potential for Further Improvement**

Knowledge Check Questions

1. Why is Recall important in a credit default scenario?

In a credit default scenario, **Recall** is extremely important because it measures the model's ability to correctly identify all actual defaulters (True Positives). The consequences of failing to identify a defaulting customer (False Negative) can be very high for a financial institution, leading to significant financial losses. While Precision (correctly identified defaulters out of all predicted defaulters) is also important to avoid denying credit to good customers, Recall helps minimize the risk of approving loans to individuals who will ultimately default.

2. What is the ROC curve and what does AUC signify?

The **ROC (Receiver Operating Characteristic) curve** is a graphical plot that illustrates the diagnostic ability of a binary classifier system as its discrimination threshold is varied. It plots the True Positive Rate (TPR, also known as Recall or Sensitivity) against the False Positive Rate (FPR, or 1 - Specificity) at various threshold settings.

AUC (Area Under the Curve) signifies the entire two-dimensional area underneath the entire ROC curve. It provides an aggregate measure of performance across all possible classification thresholds. An AUC of 1.0 indicates a perfect classifier (100% TPR, 0% FPR), while an AUC of 0.5 suggests a classifier that performs no better than random guessing.

3. Why is Sigmoid used in the output layer for binary classification?

The **Sigmoid activation function** is used in the output layer for binary classification because it squashes the input values (the output of the previous layer) into a range between 0 and 1. This output can then be interpreted as a probability. For binary classification, this probability represents the likelihood that the input belongs to the positive class (e.g., probability of default). A common threshold (e.g., 0.5) is then applied to convert these probabilities into binary class predictions.

4. What advantage does ANN provide over traditional ML algorithms?

Artificial Neural Networks (ANNs) provide several advantages over traditional ML algorithms, especially with complex data:

1. **Automatic Feature Learning**: ANNs can automatically learn and extract relevant features from raw data, reducing the need for manual feature engineering, which is often a time-consuming and expertise-dependent process in traditional ML.
2. **Handling Non-Linearity**: ANNs are highly effective at modeling complex non-linear relationships between inputs and outputs, often outperforming linear models like Logistic Regression in such scenarios.
3. **Scalability with Data**: With increasing amounts of data, ANNs (especially deeper architectures) can continue to improve their performance, whereas many traditional ML algorithms hit a plateau.
4. **Adaptability**: ANNs can be adapted and fine-tuned for a variety of tasks (e.g., image recognition, natural language processing) and can be used for transfer learning, leveraging pre-trained models.

5. Why is feature scaling important for neural networks?

Feature scaling is important for neural networks for several reasons:

1. **Faster Convergence**: Many optimization algorithms used in neural networks (e.g., Gradient Descent) converge much faster when features are on a similar scale. If features have vastly different ranges, the cost function will have elongated contours, leading to slower convergence and potentially getting stuck in local minima.
2. **Avoid Dominance by Large Values**: Features with larger numerical values might dominate the learning process, even if they are not more important functionally. Scaling

ensures that all features contribute proportionally to the distance calculations and gradient updates.

3. **Improved Weight Initialization**: Proper scaling helps with weight initialization, ensuring that activation functions are not saturated too early in the training process.
4. **Regularization Benefits**: Some regularization techniques, like L1/L2 regularization, perform better when features are scaled, as they assume that feature scales are comparable.